

Object Oriented Programming

The most common way to **ORGANIZE**, code is to *combine two concepts*. Objects, and functions. Into something called **CLASSES**.

Remember how objects *are dictionaries, but instead*; we use them to describe "Objects", as in nouns like people, places, things. We only gave those objects **DESCRIPTIVE** things which were called attributes ; like a name, an ID, or color.

Remember, how in the last part *we made functions*. **Checking out**, and **View Cart**. The PERSON object would be the one who would use these functions. So we can **ADD** these functions to the person object.

CLASSES are just a nice way to **group an object and their functions**.

- The nuance is, *classes let you CREATE an object*. Look at this example to understand, Then we'll cover how to actually do classes later.

```
class Person:
    def __init__(self, name):
        self.name = name
    def greet(self):
        return ("Hello,", self.name, "!")
```

- Let's say I want to *create a person*. I'm going to use the function Person to do that.
- **But**, a person needs to have *some attributes* we give it.
 - Those are **described** in the __init__. Meaning, we need to give this Person function an argument to take for it's name.

```
Alice = Person("Alice")
```

- Now, we have an object; we saved it as the variable **Alice**.
 - What we **can do now**, is use that function "greet"
 - We use something called a DOT OPERATOR;
 - *WHICH IS*; **Alice.greet()** ; like **objectName.functionName()**
-

Let's dissect a real example,

```
class Person():  
    def __init__(self, name, id, discount, cart):  
        self.name = name  
        self.id = id  
        self.discount = discount  
        self.cart = cart
```

What we're doing first is called an *initialize function*. You know how we created an object in the last page.

- This describes what attributes we need to create our object.

But, then what is that "self"?

- Notice how inside the function, it's setting `self.attribute = attribute`
 - That self, which is literally just the object itself; (Like Alice)
 - self is the object, assume Alice is our person, and name is it's attribute.
 - self.name = name, is the SAME AS, Alice.name = "Alice" (which is the argument given when you make the Alice object.)
-

Now, normal functions are mostly the same | See NEW vs OLD

```
def view_subtotal(self): # The CLASS version of the function  
    total = 0  
    for item in self.cart:  
        for price in item.values():  
            total += price  
    return total
```

VS

```
def calculate_cart_total(cart): # The FUNCTION version of the function  
    total = 0  
    for item in cart:  
        for price in item.values():  
            total += price  
    return total
```

The difference is here, `cart` gets replaced by `self.cart`. Since we already have the `self`, i.e. our person we don't need any other arguments

- The "self" already knows it's cart, so we can GRAB the cart from inside there.

```
from uuid import uuid4

class Person():
    def __init__(self, name, id, discount, cart):
        self.name = name
        self.id = id
        self.discount = discount
        self.cart = cart

    def view_subtotal(self):
        total = 0
        for item in self:
            for price in item.values():
                total += price
        return total

    def view_cart(self):
        for item in self.cart:
            print(item)

    def view_products(self, products):
        for product in products:
            print(product)

    def add_to_cart(self, product_choice, product_list):
        price = product_list[product_choice]
        self.cart.append({product_choice: price})
```

Here's the **entire code**.

- You don't need to be able to replicate it.
 - Just Classes allow **you to create objects**, *Person is an object*, and a class is a good way to organize objects into their **functions and attributes**.
-

GitHub on the side has the full code for EACH section, but that's it for this lab.